

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO

CENTRO TECNOLÓGICO

COLEGIADO DO CURSO DE CIÊNCIA DA COMPUTAÇÃO

Aurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

Vitória, ES

2026

Aurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

Anteprojeto apresentado ao Colegiado do Curso de Ciência da Computação do Centro Tecnológico da Universidade Federal do Espírito Santo, como requisito para aprovação na Disciplina Projeto de Graduação I.

Universidade Federal do Espírito Santo – UFES

Centro Tecnológico

Colegiado do Curso de Ciência da Computação

Orientador: Prof. Dr. João Paula Almeida

Coorientador: Prof. Dr. Ciclano de Tal

Vitória, ES

2026

1 Introdução

Sistemas de *Retrieval-Augmented Generation* (RAG) tornaram-se uma das abordagens mais utilizadas para permitir que modelos de linguagem de grande porte (*Large Language Models* – LLMs) respondam perguntas com base em uma base de conhecimento externa e específica de domínio, reduzindo problemas como respostas desatualizadas ou alucinadas . Na abordagem tradicional de RAG, o texto é dividido em fragmentos (*chunks*), convertido em vetores por meio de *embeddings* e recuperado por similaridade semântica no momento da consulta. Essa estratégia, no entanto, apresenta limitações quando o domínio de conhecimento possui uma estrutura fortemente relacional, na qual a resposta correta depende de conectar informações dispersas em diferentes trechos do texto – situação recorrente em textos legislativos, nos quais artigos remetem uns aos outros, estabelecem exceções e são complementados por súmulas e jurisprudência.

O *framework* GraphRAG, desenvolvido pela Microsoft , propõe uma alternativa a essa limitação: em vez de recuperar apenas fragmentos de texto semanticamente similares, o GraphRAG constrói um grafo de conhecimento a partir do corpus de entrada, extraindo entidades e relações por meio de um LLM, agrupando-as em comunidades hierárquicas e gerando resumos para essas comunidades. Esse processo permite tanto buscas locais, focadas em entidades específicas, quanto buscas globais, capazes de sintetizar informação distribuída por múltiplas partes do corpus – uma capacidade particularmente relevante para a análise de legislação.

Este trabalho tem como contexto a aplicação do GraphRAG à legislação do Código de Defesa do Consumidor (CDC, Lei n.º 8.078/1990). O escopo do trabalho está delimitado à construção de uma prova de conceito (*Proof of Concept* – PoC): um sistema que indexa o texto do CDC utilizando o GraphRAG, armazenando as informações como um grafo de entidades e relações juridicamente relevantes, e que disponibiliza uma aplicação web para (i) visualização interativa do grafo de conhecimento construído e (ii) realização de consultas em linguagem natural, utilizando as diferentes modalidades de busca oferecidas pelo GraphRAG (local, global, *drift* e *basic*). Ficam fora do escopo deste trabalho a extensão do sistema a outras legislações e a disponibilização do sistema em ambiente de produção – delimitações necessárias para viabilizar a conclusão do trabalho dentro do tempo hábil de um Trabalho de Conclusão de Curso.

1.1 Motivação e Justificativa

A motivação deste trabalho parte do interesse em investigar, na prática, as limitações dos sistemas de RAG tradicionais ao lidar com textos que possuem forte densidade de

relações internas, como é o caso da legislação. Textos jurídicos raramente podem ser interpretados de forma isolada: um único artigo frequentemente depende de definições estabelecidas em outros artigos, está sujeito a exceções previstas em dispositivos distintos e é interpretado à luz de súmulas e entendimentos consolidados. Uma abordagem de RAG baseada apenas em similaridade vetorial entre fragmentos de texto tende a perder essas conexões, recuperando trechos isolados que, embora semanticamente relacionados à pergunta, não contemplam a cadeia de relações necessária para uma resposta completa e juridicamente coerente.

O Código de Defesa do Consumidor foi escolhido como domínio de aplicação por três razões principais. Primeiro, trata-se de uma legislação de grande relevância social e de ampla aplicação no cotidiano da sociedade brasileira, tornando-se um domínio de interesse para a investigação e avaliação da abordagem proposta. Segundo, o CDC possui uma estrutura de tamanho gerenciável para os fins de uma PoC, permitindo iterações de desenvolvimento e ajuste do sistema em tempo hábil, ao mesmo tempo em que preserva a complexidade relacional típica de textos legislativos. Terceiro, o crescimento do interesse em soluções de acesso à justiça apoiadas por inteligência artificial reforça a relevância prática de investigar sistemas de recuperação de informação mais adequados a esse domínio.

Ainda que sistemas de RAG aplicados ao domínio jurídico já tenham sido explorados na literatura, a maior parte desses trabalhos utiliza abordagens de recuperação vetorial tradicional. Este trabalho busca contribuir com essa lacuna, investigando de forma prática se a abordagem em grafo do GraphRAG traz ganhos perceptíveis de qualidade e completude nas respostas, quando comparada à abordagem vetorial tradicional, em um domínio legislativo de escopo delimitado.

1.2 Objetivos

Objetivo Geral

Desenvolver uma prova de conceito de um sistema de *Retrieval-Augmented Generation* baseado em grafo de conhecimento (GraphRAG) para indexação e consulta da legislação do Código de Defesa do Consumidor, disponibilizando uma aplicação web que permita a visualização do grafo de conhecimento construído e a realização de consultas em diferentes modalidades de busca.

Objetivos Específicos

- Revisar a literatura sobre sistemas de *Retrieval-Augmented Generation*, RAG baseado em grafos de conhecimento e aplicações de Processamento de Linguagem Natural no domínio jurídico.

- Definir uma taxonomia de entidades e relações adequada à estrutura do Código de Defesa do Consumidor, a ser utilizada no processo de extração de grafo.
- Implementar o *pipeline* de indexação do GraphRAG sobre o texto do CDC, compreendendo a extração de entidades e relações, a construção do grafo e a detecção de comunidades.
- Desenvolver uma aplicação web para visualização interativa do grafo de conhecimento gerado e realização de consultas em linguagem natural, permitindo a utilização das diferentes modalidades de busca disponibilizadas pelo GraphRAG (*local*, *global*, *drift* e *basic*).
- Avaliar comparativamente a qualidade e a completude das respostas obtidas por meio do GraphRAG em relação a uma abordagem de RAG vetorial tradicional, utilizada como *baseline*.

1.3 Método de Desenvolvimento do Trabalho

O desenvolvimento deste trabalho caracteriza-se como uma pesquisa aplicada, de natureza tecnológica, cujo produto final é uma prova de conceito funcional. O trabalho será conduzido por meio das seguintes etapas metodológicas:

- **Revisão bibliográfica:** levantamento de referenciais teóricos sobre RAG, RAG baseado em grafos (com foco no *framework* GraphRAG) e extração de entidades e relações via LLMs.
- **Preparação do corpus:** obtenção e pré-processamento do texto do Código de Defesa do Consumidor, incluindo a definição da granularidade de segmentação (*chunking*) do texto para a etapa de indexação.
- **Definição da taxonomia do domínio:** especificação dos tipos de entidade e relação relevantes para a legislação em análise (por exemplo, artigo, direito, obrigação, sanção, relação de consumo), a serem utilizados para orientar a extração de grafo realizada pelo LLM.
- **Configuração do ambiente de indexação:** montagem do ambiente de execução do GraphRAG, incluindo a definição do(s) modelo(s) de linguagem utilizados para as etapas de extração, resumo e geração de respostas, executados em ambiente conteinerizado para garantir reprodutibilidade.
- **Execução e ajuste do *pipeline* de indexação:** execução do *pipeline* de indexação do GraphRAG sobre o corpus, seguida de validação manual da qualidade das entidades

e relações extraídas, com ajustes iterativos de configuração e de *prompts* até que a extração atinja um nível satisfatório de cobertura e coerência.

- **Desenvolvimento da aplicação web:** implementação de um *backend* responsável por expor as funcionalidades de consulta do GraphRAG (busca local, global, *drift* e *basic*) e de um *frontend* para visualização interativa do grafo de conhecimento e para a submissão de consultas em linguagem natural.
- **Avaliação:** definição de um conjunto de perguntas de teste sobre o CDC e comparação qualitativa das respostas obtidas pelas diferentes modalidades de busca do GraphRAG frente a uma abordagem de RAG vetorial tradicional, utilizada como *baseline* de comparação.

1.4 Cronograma

As atividades previstas para o desenvolvimento deste trabalho, alinhadas aos objetivos específicos apresentados na Seção 1.2, são descritas a seguir:

- **Atividade 1 – Revisão bibliográfica:** levantamento e leitura de referenciais teóricos sobre RAG, GraphRAG e PLN aplicada ao domínio jurídico, culminando na escrita do referencial teórico do trabalho.
- **Atividade 2 – Definição da taxonomia e preparação do corpus:** definição dos tipos de entidade e relação do domínio, e preparação do texto do CDC para indexação.
- **Atividade 3 – Configuração do ambiente de indexação:** montagem do ambiente conteinerizado e definição dos modelos de linguagem utilizados.
- **Atividade 4 – Execução e ajuste da indexação:** execução iterativa do *pipeline* de indexação do GraphRAG, com validação e ajuste da qualidade da extração do grafo.
- **Atividade 5 – Desenvolvimento do *backend*:** implementação da camada de *backend* responsável por expor as consultas ao GraphRAG.
- **Atividade 6 – Desenvolvimento do *frontend*:** implementação da interface web de visualização do grafo e de consulta.
- **Atividade 7 – Avaliação comparativa:** definição do conjunto de perguntas de teste e comparação entre as modalidades de busca do GraphRAG e o *baseline* vetorial.
- **Atividade 8 – Escrita e revisão do TCC:** consolidação dos resultados e redação final do documento do trabalho.

Tabela 1 – Cronograma de execução das atividades

	Mês 1	**Mês 2**	**Mês 3**	**Mês 4**	**Mês 5**	**Mês 6**
Atividade 1	X	X				
Atividade 2		X	X			
Atividade 3		X				
Atividade 4		X	X	X		
Atividade 5				X	X	
Atividade 6				X		
Atividade 7					X	X
Atividade 8	X	X	X	X	X	X

2 Fundamentação Teórica e Tecnologias Utilizadas

Este capítulo apresenta os fundamentos teóricos e as tecnologias que sustentam o desenvolvimento deste trabalho. A Seção 2.1 introduz o conceito de *Retrieval-Augmented Generation* (RAG) e discute as limitações da abordagem vetorial tradicional. A Seção 2.2 apresenta o RAG baseado em grafos de conhecimento, com ênfase na arquitetura, no *pipeline* de indexação e nas modalidades de consulta implementadas pela biblioteca GraphRAG, desenvolvida pela Microsoft, que será utilizada como núcleo deste trabalho. A Seção 2.3 discute a abordagem de RAG vetorial que será utilizada como *baseline* de comparação. Por fim, a Seção 2.4 apresenta as tecnologias auxiliares escolhidas para o desenvolvimento da aplicação web que compõe a prova de conceito.

2.1 *Retrieval-Augmented Generation* (RAG)

Modelos de linguagem de grande porte (*Large Language Models* – LLMs) são treinados sobre grandes volumes de texto até uma determinada data de corte, o que os torna limitados quando confrontados com perguntas que exigem conhecimento posterior a esse treinamento ou conhecimento específico de um domínio não representado de forma significativa nos dados de treinamento. Além disso, LLMs estão sujeitos ao fenômeno de *alucinação*, no qual o modelo gera respostas plausíveis, porém factualmente incorretas ou não fundamentadas em nenhuma fonte real.

O paradigma de *Retrieval-Augmented Generation* foi proposto como forma de mitigar essas limitações, combinando um mecanismo de recuperação de informação (*retrieval*) a um modelo de geração de texto. Em vez de depender exclusivamente do conhecimento paramétrico codificado nos pesos do modelo durante o treinamento, um sistema RAG recupera, no momento da consulta, os trechos mais relevantes de uma base de conhecimento externa e os inclui no contexto (*prompt*) fornecido ao LLM, que então gera a resposta com base nesse conteúdo recuperado. Esse mecanismo permite atualizar o conhecimento do sistema sem a necessidade de re-treinar o modelo, além de tornar as respostas passíveis de rastreamento até sua fonte.

Na implementação mais comum de RAG, conhecida como RAG vetorial ou RAG ingênuo (*naive RAG*), o processo ocorre em duas etapas principais. Na etapa de indexação, os documentos da base de conhecimento são divididos em fragmentos menores (*chunks*), e cada fragmento é convertido em um vetor numérico de alta dimensão por meio de um modelo de *embeddings*, que captura sua representação semântica. Esses vetores são

armazenados em um banco de dados vetorial. Na etapa de consulta, a pergunta do usuário é igualmente convertida em vetor, e os fragmentos armazenados com maior similaridade de *cosseno* em relação a esse vetor são recuperados e inseridos no contexto do LLM.

Embora essa abordagem seja eficaz em diversos cenários, ela apresenta limitações relevantes para o presente trabalho. A recuperação por similaridade vetorial opera sobre fragmentos de texto de forma isolada, sem representar explicitamente as relações estruturais entre diferentes partes do corpus. Em domínios como o jurídico, nos quais um artigo de lei frequentemente depende de definições, exceções e remissões estabelecidas em outros artigos, essa limitação pode levar à recuperação de fragmentos parcialmente relevantes, mas insuficientes para compor uma resposta completa. Adicionalmente, o RAG vetorial tradicional apresenta dificuldade em responder perguntas de natureza global ou holística – por exemplo, "quais são os principais direitos do consumidor previstos no código?" –, uma vez que a resposta demandaria a síntese de informações dispersas por praticamente todo o corpus, e não a recuperação de um pequeno conjunto de fragmentos localizados.

2.2 RAG Baseado em Grafos de Conhecimento (GraphRAG)

Como alternativa às limitações do RAG vetorial, diferentes abordagens têm proposto a estruturação da base de conhecimento como um grafo, no qual nós representam entidades do domínio e arestas representam as relações entre elas. Essa representação preserva explicitamente a estrutura relacional do corpus e permite estratégias de recuperação capazes de percorrer múltiplos saltos (*multi-hop*) entre entidades relacionadas, além de viabilizar a geração de resumos hierárquicos sobre agrupamentos de entidades (comunidades), o que favorece a resposta a perguntas de natureza global.

2.2.1 O *framework* GraphRAG da Microsoft

Este trabalho utiliza o GraphRAG, biblioteca de código aberto desenvolvida pela Microsoft Research , disponibilizada como pacote Python (`graphrag`) e distribuída sob licença MIT. O GraphRAG propõe um método estruturado e hierárquico de RAG que, diferentemente da abordagem vetorial ingênua, extrai um grafo de conhecimento a partir do texto bruto por meio de um LLM, constrói uma hierarquia de comunidades sobre esse grafo e gera resumos textuais para cada comunidade, utilizando essas estruturas – e não apenas fragmentos de texto isolados – como base para a geração de respostas.

2.2.1.1 *Pipeline* de indexação

A indexação de um corpus no GraphRAG é organizada como um *pipeline* de *workflows* executados sequencialmente, cada um responsável por uma etapa específica do processamento:

- **Carregamento e segmentação do texto:** os documentos de entrada são carregados e divididos em unidades de texto (*text units*) de tamanho configurável, medido em número de *tokens*, com sobreposição (*overlap*) entre unidades consecutivas para preservar contexto nas bordas dos fragmentos.
- **Extração do grafo:** para cada unidade de texto, o LLM configurado é utilizado para extrair entidades e relações entre elas, segundo uma taxonomia de tipos de entidade fornecida na configuração (por exemplo, organização, pessoa, evento – no caso deste trabalho, adaptada para o domínio jurídico). O processo suporta múltiplas "passadas" de extração sobre a mesma unidade de texto (parâmetro *gleanings*), nas quais o modelo é solicitado a identificar entidades adicionais que não tenham sido capturadas na primeira passagem, aumentando a cobertura da extração.
- **Resumo de descrições:** quando a mesma entidade é mencionada em diferentes unidades de texto, suas descrições parciais são consolidadas em uma única descrição coerente, também com auxílio do LLM.
- **Detecção de comunidades:** o grafo de entidades e relações extraído é submetido a um algoritmo de detecção de comunidades – o GraphRAG utiliza, por padrão, o algoritmo de Leiden –, que particiona o grafo em grupos hierárquicos de entidades fortemente conectadas entre si, formando diferentes níveis de granularidade (de comunidades menores e mais específicas a comunidades maiores e mais abrangentes).
- **Geração de relatórios de comunidade:** para cada comunidade identificada, o LLM gera um relatório textual que resume as entidades, relações e temas predominantes daquele agrupamento. Esses relatórios são, posteriormente, a base para a busca global.
- **Geração de *embeddings*:** por fim, são gerados vetores de *embedding* para as descrições de entidades, para as unidades de texto originais e para os relatórios de comunidade, armazenados em um banco de dados vetorial (por padrão, LanceDB) para uso nas etapas de consulta que combinam busca vetorial com a estrutura do grafo.

Toda a saída do *pipeline* de indexação – entidades, relações, comunidades, relatórios e unidades de texto – é persistida em arquivos no formato *parquet*, o que facilita tanto o consumo por outras aplicações (como a aplicação web deste trabalho) quanto a inspeção manual dos dados durante o desenvolvimento.

2.2.1.2 Modalidades de consulta

Uma das características centrais do GraphRAG, e de particular interesse para este trabalho, é a disponibilização de diferentes modalidades de busca, cada uma adequada a um tipo distinto de pergunta:

- **Busca local (*Local Search*):** indicada para perguntas pontuais, focadas em entidades específicas (por exemplo, "o que diz o artigo 18 do CDC sobre vício do produto?"). Combina dados estruturados do grafo (entidades relacionadas, suas descrições e relações diretas) com trechos de texto não estruturado associados a essas entidades, compondo um contexto rico e específico para o LLM gerador.
- **Busca global (*Global Search*):** indicada para perguntas amplas ou holísticas, que exigem sintetizar informação distribuída por diversas partes do corpus (por exemplo, "quais são os principais direitos do consumidor previstos no CDC?"). Opera sobre os relatórios de comunidade gerados na indexação, utilizando uma estratégia de *map-reduce*: cada relatório de comunidade é processado individualmente para gerar uma resposta parcial (*map*), e essas respostas parciais são então combinadas em uma resposta final consolidada (*reduce*).
- **Busca *DRIFT* (*Dynamic Reasoning and Inference with Flexible Traversal*):** combina elementos das buscas local e global de forma iterativa. Parte de uma resposta inicial abrangente (*primer*), gerada de forma semelhante à busca global, e a refina por meio de sucessivas perguntas de acompanhamento (*follow-ups*), respondidas via busca local, permitindo maior profundidade de raciocínio sobre a pergunta original.
- **Busca básica (*Basic Search*):** realiza busca vetorial simples sobre as unidades de texto originais, sem envolver a estrutura do grafo ou os relatórios de comunidade. Funciona, na prática, como uma implementação de RAG vetorial tradicional dentro do próprio GraphRAG, compartilhando o mesmo corpus indexado, modelo de *embeddings* e modelo gerador utilizados nas demais modalidades.

2.2.1.3 Aspectos de configuração e execução

Em termos de implementação, a biblioteca GraphRAG é configurada por meio de um arquivo `settings.yaml`, no qual são definidos os modelos de linguagem utilizados (um para completions/geração e outro para *embeddings*), parâmetros de segmentação do texto, a taxonomia de tipos de entidade a ser extraída, parâmetros de particionamento de comunidades e as configurações específicas de cada modalidade de consulta. A comunicação com os provedores de modelos de linguagem é abstraída pela biblioteca *LiteLLM*, o que permite configurar o GraphRAG para utilizar tanto provedores comerciais baseados

em API (como OpenAI, Google Gemini ou OpenRouter) quanto modelos executados localmente por meio de ferramentas como o Ollama – flexibilidade explorada neste trabalho para viabilizar a execução do sistema sem dependência obrigatória de custos de API. A biblioteca disponibiliza ainda um utilitário de ajuste automático de *prompts* (`graphrag prompt-tune`), que gera exemplos de extração adaptados ao vocabulário e à estrutura do corpus de entrada, reduzindo a necessidade de engenharia manual de *prompts* para domínios específicos como o jurídico.

2.3 RAG Vetorial como Baseline de Comparação

Um dos objetivos específicos deste trabalho consiste em avaliar comparativamente a qualidade e a completude das respostas obtidas por meio do GraphRAG frente a uma abordagem de RAG vetorial tradicional.Para isso, será utilizada como baseline a própria modalidade de **busca básica** do GraphRAG (Seção 2.2.1.2).

Como essa modalidade realiza busca vetorial pura sobre o mesmo corpus, com o mesmo modelo de *embeddings* e o mesmo modelo gerador utilizados nas demais modalidades, ela isola a variável de interesse do experimento – a presença ou ausência da estrutura de grafo –, eliminando diferenças de configuração que poderiam confundir a comparação.

2.4 Tecnologias de Apoio ao Desenvolvimento da Aplicação Web

Como o foco principal deste trabalho é a investigação da abordagem GraphRAG aplicada ao domínio jurídico, e não o desenvolvimento de uma aplicação web sofisticada, a escolha das tecnologias de apoio priorizou simplicidade de implementação e produtividade, de modo a permitir que a maior parte do esforço de desenvolvimento seja dedicada à indexação, ao ajuste do grafo de conhecimento e à avaliação dos resultados.

2.4.1 *Backend*

O backend será implementado em Python utilizando o framework FastAPI, devido à sua compatibilidade direta com a biblioteca GraphRAG, também desenvolvida em Python. Essa integração permite invocar diretamente as funcionalidades de consulta do framework, além de oferecer suporte a operações assíncronas e geração automática da documentação da API.

O *backend* será responsável por expor endpoints REST para: (i) submissão de consultas em linguagem natural, com seleção da modalidade de busca desejada; (ii) obtenção dos dados do grafo de conhecimento (entidades, relações e comunidades) para consumo pela interface de visualização.

2.4.2 *Frontend*

Para a interface web, propõe-se o uso de **React** com **TypeScript**, construído sobre a ferramenta de *build* **Vite**, combinação amplamente utilizada e bem documentada para o desenvolvimento de interfaces modernas de forma ágil. Para a estilização, propõe-se o uso do **Tailwind CSS** em conjunto com uma biblioteca de componentes prontos (como *shadcn/ui*), o que permite obter uma interface visualmente cuidada com baixo esforço de design, alinhado à prioridade de manter a aplicação simples, funcional e esteticamente agradável, sem que essa camada consuma esforço desproporcional do trabalho.

Para a visualização interativa do grafo de conhecimento – um dos entregáveis centrais da aplicação –, propõe-se o uso de uma biblioteca especializada em visualização de grafos sobre React, como a **react-force-graph** (baseada em simulação de forças, *force-directed layout*) ou o **Cytoscape.js**, ambas amplamente utilizadas para esse tipo de visualização e capazes de lidar com interações como zoom, arraste de nós e exibição de detalhes (descrição, tipo, relações) ao selecionar uma entidade. A escolha final entre essas alternativas será feita durante a fase de desenvolvimento, com base em testes de desempenho sobre o volume de dados do grafo gerado para o CDC completo.

2.4.3 Infraestrutura e Execução

Todo o ambiente do projeto – incluindo o *backend*, o *frontend*, o modelo de linguagem local (quando utilizado, por meio do Ollama) e o *pipeline* de indexação do GraphRAG – será orquestrado por meio de contêineres **Docker**, com definição dos serviços em um arquivo `docker-compose.yml`. Essa escolha garante a reprodutibilidade do ambiente de desenvolvimento e de execução, facilita a documentação do processo de instalação no corpo do trabalho e permite que o sistema seja executado de forma consistente independentemente do sistema operacional do avaliador.

Referências